

BTS Services Informatiques aux Organisations (SIO)
Session 2025

Nom du candidat : Da Silva Peixoto

Prénom du candidat : Gabriel

Parcours : BTS SIO option SLAM

Fiche de réalisation professionnelle N° 12

**Développement d'une application web monolithique
de gestion hôtelière**

Période de réalisation : 2025-2026

Type de réalisation : Durant la formation

Mode de réalisation : Individuelle

Sommaire

Contexte organisationnel	3
Problématique	3
Choix de la solution mise en œuvre	3
Résolution	3
Conclusions	4
Compétences	4

CONTEXTE ORGANISATIONNEL

Dans le cadre de ma formation en BTS SIO, j'ai conçu et développé un Système d'Information dédié à la gestion d'un établissement hôtelier. Ce projet avait pour but de numériser et de centraliser les opérations quotidiennes de l'hôtel. J'ai été chargé de développer une application permettant de gérer simultanément le catalogue des chambres, le fichier des clients et le suivi des réservations. Contrairement à une approche séparant une API et un client lourd ou web, j'ai opté pour une architecture monolithique développée en Node.js, utilisant Express comme framework et EJS (Embedded JavaScript) comme moteur de templating pour générer les vues.

PROBLÉMATIQUE

Le développement d'une application web monolithique pose le défi de la cohabitation entre la logique métier complexe (Back-End) et la génération de l'interface utilisateur (Front-End) au sein du même projet. La difficulté principale résidait dans le rendu dynamique et sécurisé des données : il fallait afficher en temps réel la disponibilité des chambres, traiter les formulaires de réservation complexes, et gérer les sessions des utilisateurs (clients et administrateurs) sans l'appui d'un framework Front-End autonome (comme React ou Angular). Il était crucial de structurer le code de manière rigoureuse pour éviter que la logique de traitement des données de l'hôtel ne s'emmêle avec le code HTML, ce qui aurait rendu l'application instable et difficilement maintenable.

CHOIX DE LA SOLUTION MISE EN ŒUVRE

Pour répondre à ce besoin, j'ai choisi de structurer mon application Node.js/Express selon le motif d'architecture logicielle MVC (Modèle-Vue-Contrôleur). Mon choix s'est porté sur le moteur de template EJS pour les "Vues". EJS permet d'injecter directement des données dynamiques (issues de la base de données de l'hôtel) dans le code HTML (Server-Side Rendering) de manière très intuitive. Ce choix technologique offrait trois avantages : une grande rapidité d'exécution côté serveur, un excellent référencement naturel (SEO) puisque les pages sont livrées pré-calculées au navigateur, et une centralisation de la sécurité (les validations de formulaires et les vérifications de session se faisant exclusivement sur le serveur Node.js avant tout affichage).

RÉSOLUTION

La mise en œuvre de l'application s'est déroulée en plusieurs étapes méthodiques :

- **Structuration et Modélisation** : J'ai initié le projet Node.js et défini mes Modèles de données (Chambres, Clients, Réservations) pour interagir avec la base de données.
- **Développement des Contrôleurs et des Routes** : J'ai créé les routes Express pour intercepter les requêtes HTTP. Par exemple, une route `GET /chambres` récupère la liste des chambres disponibles via le contrôleur, tandis qu'une route `POST /reservation` traite la soumission d'un formulaire de réservation.
- **Intégration du moteur EJS** : J'ai développé les vues en y intégrant la syntaxe EJS. Cela m'a permis de créer des boucles dynamiques pour afficher le catalogue des chambres (nom, prix, photo) et des conditions pour adapter le menu de navigation (afficher "Mon Compte" si le client a une session active).
- **Sécurisation et Sessions** : J'ai mis en place un système de sessions côté serveur (avec `express-session`) pour authentifier les clients, protéger l'accès à leur historique de réservation, et restreindre l'accès au tableau de bord (Back-Office) aux seuls administrateurs de l'hôtel.

CONCLUSIONS

Le choix d'une architecture monolithique avec Node.js et EJS s'est révélé extrêmement pertinent pour ce projet de gestion hôtelière. Cette approche a permis de livrer une application performante, avec des temps de chargement très courts, tout en garantissant une haute sécurité des transactions puisque toute la logique de validation des réservations est verrouillée côté serveur. Ce projet m'a permis de consolider mes compétences sur le cycle de vie complet d'une requête HTTP, de l'interaction avec la base de données jusqu'au rendu final du HTML dynamique envoyé au client.

COMPÉTENCES

B1.1.1 Mettre en place et vérifier les niveaux d'habilitation associés à un service

B1.1.7 Vérifier le respect des règles d'utilisation des ressources numériques

B.1.5.1 Déployer un service